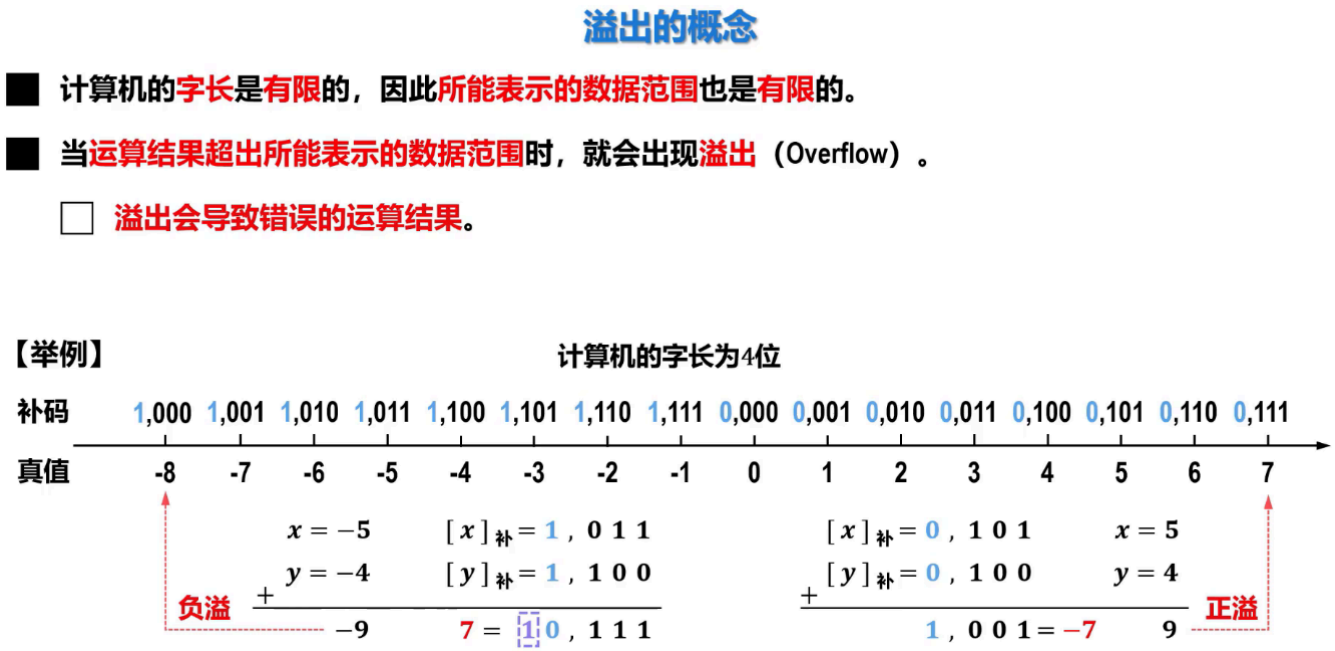
一、溢出的概念

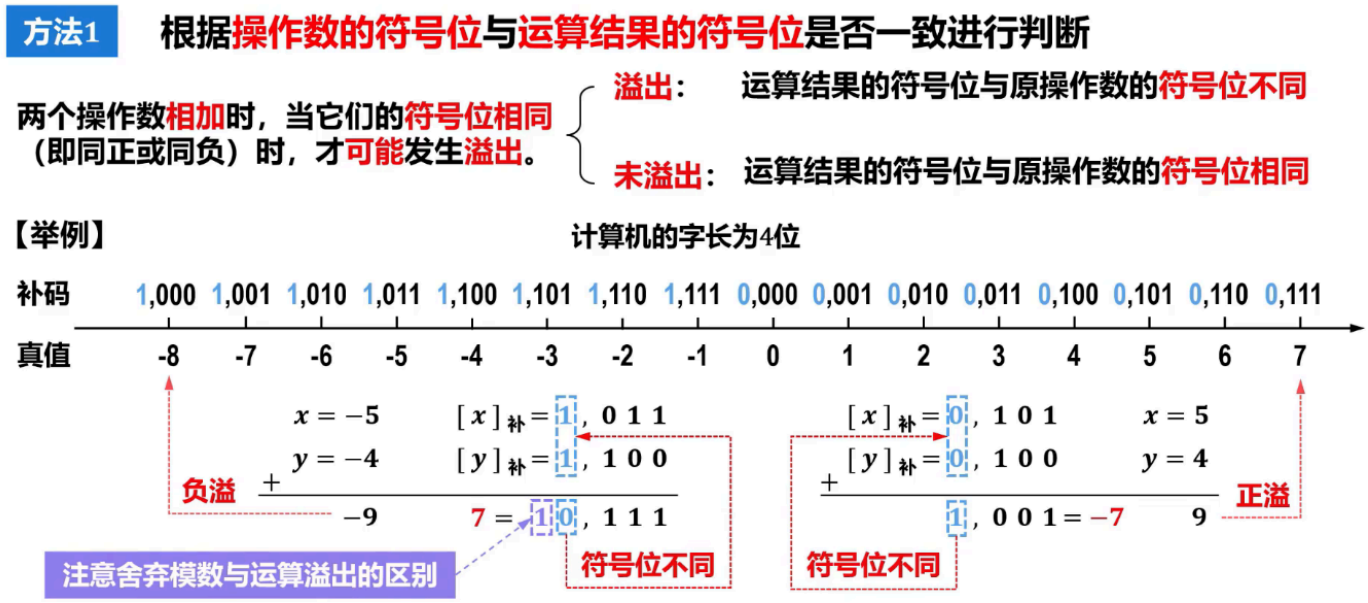


注：只有两个操作数符号相同时，才可能发生溢出

二、溢出检测的方法

2.1 法1：检测操作数与运算数的符号位是否一致

2.1.1 *规则*



注：操作数的符号，指的是**实际参与运算**的操作数的符号（我们知道减法可以转换成加法[[3.4 定点数的加减运算——补码运算规则]]，所以对于x-y，转成x+(-y)，则-y的符号才是需要考虑的）

2.1.2 *例题*

【举例】若$x=+101$，$y=-011$，采用4位补码表示，判断$[x-y]_{补}$是否会溢出。

$[x]_{补}=0,\ 1\ 0\ 1$

$[-y]_{补}=0,\ 0\ 1\ 1$

$+$

$1,\ 0\ 0\ 0$

符号位不同，溢出

2.1.3 *溢出检测的逻辑表达式*:

操作数 x 的符号位记为x_{n-1}，操作数 y 的符号位记为y_{n-1}，运算结果 s 的符号位记为s_{n-1}，

溢出标志位记为OF，当OF为1时表示发生溢出。

x与y的符号位相同，与s的符号位相反 $\begin{cases} x_{n-1}\cdot y_{n-1}\cdot \overline{s}_{n-1} \\ \overline{x}_{n-1}\cdot \overline{y}_{n-1}\cdot s_{n-1} \end{cases}$

$OF=x_{n-1}\cdot y_{n-1}\cdot \overline{s}_{n-1}+\overline{x}_{n-1}\cdot \overline{y}_{n-1}\cdot s_{n-1}$

$OF=1\cdot 1\cdot \overline{1}\ +\ \overline{1}\cdot \overline{1}\cdot 1\ =0$

【例题1】若$x=-101$，$y=-011$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

x_{n-1}

$[x]_{补}=1,\ 0\ 1\ 1$

$[y]_{补}=1,\ 1\ 0\ 1$

$+\quad y_{n-1}$

$1\ 1,\ 0\ 0\ 0$

s_{n-1} 符号位相同，未溢出

符号位进位的值为模数，应舍弃

2.2 法2：检测最高数值位的进位与符号位的进位是否一致

2.2.1 规则

溢出检测

方法2 根据运算过程中最高数值位的进位与符号位的进位是否一致进行判断

不同可判定为产生溢出，相同可判定为没有产生溢出。

【举例】 若$x=5$，$y=4$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

符号位的进位 ，最高数值位的进位

进位 $0\neq 1$

$x=5\quad [x]_{补}\quad 0,\ 1\ 0\ 1$

$y=4\quad [y]_{补}\quad 0,\ 1\ 0\ 0$

$+$

9 正溢 $\quad 1,\ 0\ 0\ 1=-7$

【举例】 若$x=-5$，$y=-4$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

符号位的进位 ，最高数值位的进位

进位 $1\neq 0$

$x=-5\quad [x]_{补}\quad 1,\ 0\ 1\ 1$

$y=-4\quad [y]_{补}\quad 1,\ 1\ 0\ 0$

$+$

负溢 $-9\quad 1\ \ 0,\ 1\ 1\ 1=7$

符号位进位的值为模数，应舍弃

2.2.2 *例题*

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

【练习3】若 $x=-111$，$y=-010$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

符号位的进位 ——┐ ┌—— 最高数值位的进位

进位　　1≠0　溢出

$[x]_{补}$　　1，0 0 1

$[y]_{补}$　　1，1 1 0

\+

1　0，1 1 1

符号位进位的值为模数，应舍弃

【练习4】若 $x=+100$，$y=+011$，采用4位补码表示，判断 $[x-y]_{补}$ 是否会溢出。

符号位的进位 ——┐ ┌—— 最高数值位的进位

进位　　1=1　未溢出

$[x]_{补}$　　0，1 0 0

$[-y]_{补}$　　1，1 0 1

\+

1　0，0 0 1

符号位进位的值为模数，应舍弃

2.2.3 *溢出检测的逻辑表达式*

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

最高数值位的进位记为 C_{n-1}，符号位的进位记为 C_n

溢出标志位记为 OF，当 OF 为1时表示发生溢出。

C_{n-1} 与 C_n 不同，则产生溢出（即 OF 为1）：

写出**溢出检测**的**逻辑表达式**?

$$OF = C_{n-1} \oplus C_n$$

2.3 法3：变形补码（具有两位符号位）检测法

2.3.1 *规则*

溢出检测

方法3 利用**变形补码**（具有2位符号位的补码）的符号位进行判断

变形补码也称为**双符号补码**，具有2个符号位，其余与补码相同。

- 双符号位为 00 时，表示**正数**；
- 双符号位为 11 时，表示**负数**；
- 双符号位为 01 时，表示**正溢出**；
- 双符号位为 10 时，表示**负溢出**；

溢出判断非常直观，适合手工运算时的溢出检测，但其硬件成本高，在计算机中主要采用单符号溢出检测方案。

注意：负数变形补码转换成原码仍可以使用反码法或扫描法。

2.3.2 *例题*

溢出检测

方法3 利用**变形补码**（具有2位符号位的补码）的符号位进行判断

- 双符号位为 00 时，表示**正数**；
- 双符号位为 11 时，表示**负数**；
- 双符号位为 01 时，表示**正溢出**；
- 双符号位为 10 时，表示**负溢出**；

【练习】(1) $[x]_{补}=00.1011$，$[y]_{补}=00.0111$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(2) $[x]_{补}=11.0101$，$[y]_{补}=11.0011$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(3) $[x]_{补}=00.1011$，$[y]_{补}=11.0011$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(1)

$$
\begin{array}{rr}
[x]_{补} & 00.1011 \\
+\ [y]_{补} & 00.0111 \\
\hline
[x]_{补}+[y]_{补} & \boxed{01}.0010
\end{array}
$$

正溢出

(2)

$$
\begin{array}{rr}
[x]_{补} & 11.0101 \\
+\ [y]_{补} & 11.0011 \\
\hline
[x]_{补}+[y]_{补} & \boxed{10}.1000
\end{array}
$$

负溢出

(3)

$$
\begin{array}{rr}
[x]_{补} & 00.1011 \\
+\ [y]_{补} & 11.0011 \\
\hline
[x]_{补}+[y]_{补} & \boxed{11}.1110
\end{array}
$$

未溢出

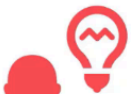
2.3.3 逻辑表达式

溢出检测

方法3 利用**变形补码**（具有2位符号位的补码）的符号位进行判断

- 双符号位为 00 时，表示**正数**；
- 双符号位为 01 时，表示**正溢出**；
- 双符号位为 11 时，表示**负数**；
- 双符号位为 10 时，表示**负溢出**；

写出**溢出检测**的**逻辑表达式**?



左起第一个符号位记为C_{n+1}

左起第二个符号位记为C_n

溢出标志位记为OF，当OF为1时表示发生溢出。

C_{n+1} 与 C_n 不同，则产生溢出（即OF为1）：

$$OF = C_{n+1} \oplus C_n$$